



Tecnológico de Monterrey

Campus Santa Fe

Parallel syntax highlighter

Julian Berges

Introducción

For this assignment we were asked to implement our evidence 1 of syntax highlighting into a parallel version. Additionally this report analyses the time complexity of two versions of a C++ syntax highlighter written in Elixir: a sequential version that processes files one at a time, and a parallel version that uses Elixir Tasks to distribute the work across all available CPU cores simultaneously.

Time complexity

Both versions of the assignment are $O(n)$ just the key difference is one performs all tasks on one thread or task and the parallel version distributes the work into the imputed number of files or the default which is 12. To run the code we use `iex highlighter.exs` and then use this command; `#FileHighlighter.measure_time(fn input, output, t -> FileHighlighter.highlight_parallel(input, output, t) end, ["cpp_files", "output", 4])`

Test 1

This test is with smaller files

The time with running sequentially. .

```
iex(3)> FileHighlighter.measure_time( fn input, output, t -> FileHighlighter.highlight_dir_parallel(input, output, t) end, ["cpp_files", "output", 1])
Done -> output/1.html
Done -> output/2.html
Done -> output/3.html
Done -> output/4.html
Done -> output/5.html
Done -> output/6.html
0.129126
iex(4)>
```

Time of parallel execution with one task per file.

```
iex(4)> FileHighlighter.measure_time( fn input, output, t -> FileHighlighter.highlight_dir_parallel(input, output, t) end, ["cpp_files", "output", 6])
Done -> output/1.html
Done -> output/2.html
Done -> output/6.html
Done -> output/4.html
Done -> output/5.html
Done -> output/3.html
0.038912
```

Test 2

Larger files of c++

Sequential execution.

```
Interactive Elixir (1.19.5) - press Ctrl+C to exit (type h() ENTER for help)
iex(1)> FileHighlighter.measure_time(fn input, output, t -> FileHighlighter.highlight_parallel(input, output, t) end, ["cpp_files", "output", 1])
Done -> output/1.html
Done -> output/2.html
Done -> output/3.html
Done -> output/4.html
Done -> output/5.html
Done -> output/6.html
5.113753
iex(2)> 
```

Parallel execution with 6 task one for each file

```
iex(2)> FileHighlighter.measure_time(fn input, output, t -> FileHighlighter.highlight_parallel(input, output, t) end, ["cpp_files", "output", 6])
Done -> output/6.html
Done -> output/4.html
Done -> output/3.html
Done -> output/5.html
Done -> output/1.html
Done -> output/2.html
3.260313
iex(3)> 
```

With 16 threads created theres not much difference.

```
iex(4)> FileHighlighter.measure_time(fn input, output, t -> FileHighlighter.highlight_parallel(input, output, t) end, ["cpp_files", "output", 16])
Done -> output/6.html
Done -> output/4.html
Done -> output/3.html
Done -> output/5.html
Done -> output/1.html
Done -> output/2.html
3.037388
```

Conclusión

With larger files and many files parallel should stand the test of efficiency against sequential executions. The moral dilemma is the computational power in this occasion, how much computational power would be enough to satisfy our needs as a society. The more resources we source into technology the bigger the need for computational power in which we will stall the progress.